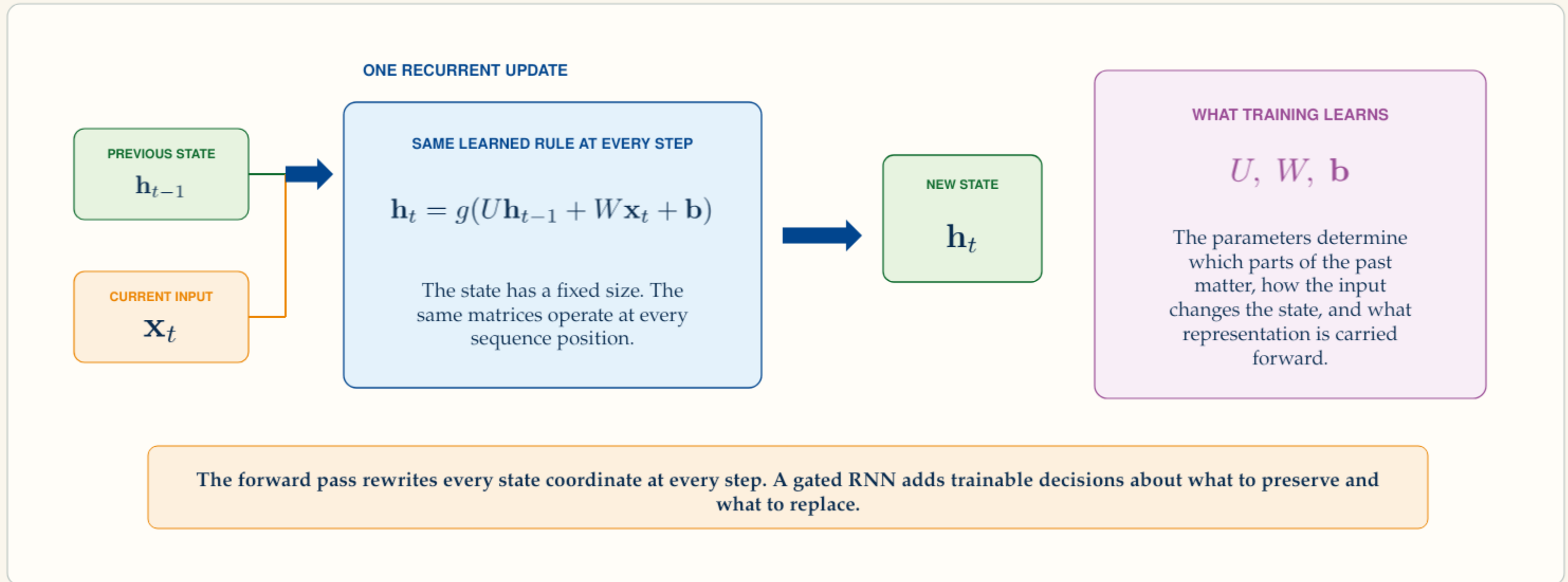


Vanilla RNN: A Trainable State Update

The forward pass assumes that one shared recurrent rule can carry information through a sequence.

THE ARCHITECTURE PROPOSES A MEMORY MECHANISM



The GRU turns state preservation and replacement into context-dependent learned decisions.

A Full GRU Makes Two Separate Decisions

The reset gate controls candidate formation; the update gate controls how the state is blended.

FOUR COMPUTATIONS AT EACH TIME STEP

- 1 Read gate** $\Gamma_{r,t} = \sigma(W_r[h_{t-1}; x_t] + b_r)$
- 2 Candidate** $\tilde{h}_t = \tanh(W_h[\Gamma_{r,t} \odot h_{t-1}; x_t] + b_h)$
- 3 Write gate** $\Gamma_{u,t} = \sigma(W_u[h_{t-1}; x_t] + b_u)$
- 4 New state** $h_t = (1 - \Gamma_{u,t}) \odot h_{t-1} + \Gamma_{u,t} \odot \tilde{h}_t$

$h_{t-1}, \tilde{h}_t, h_t \in \mathbb{R}^d, \quad \Gamma_{r,t}, \Gamma_{u,t} \in (0, 1)^d$
Both gates have the same vector length as the recurrent state.

Two Gates, Two Locations



What should the candidate read from the previous state?



How much of the candidate should be written?

Both decisions depend on the current context.

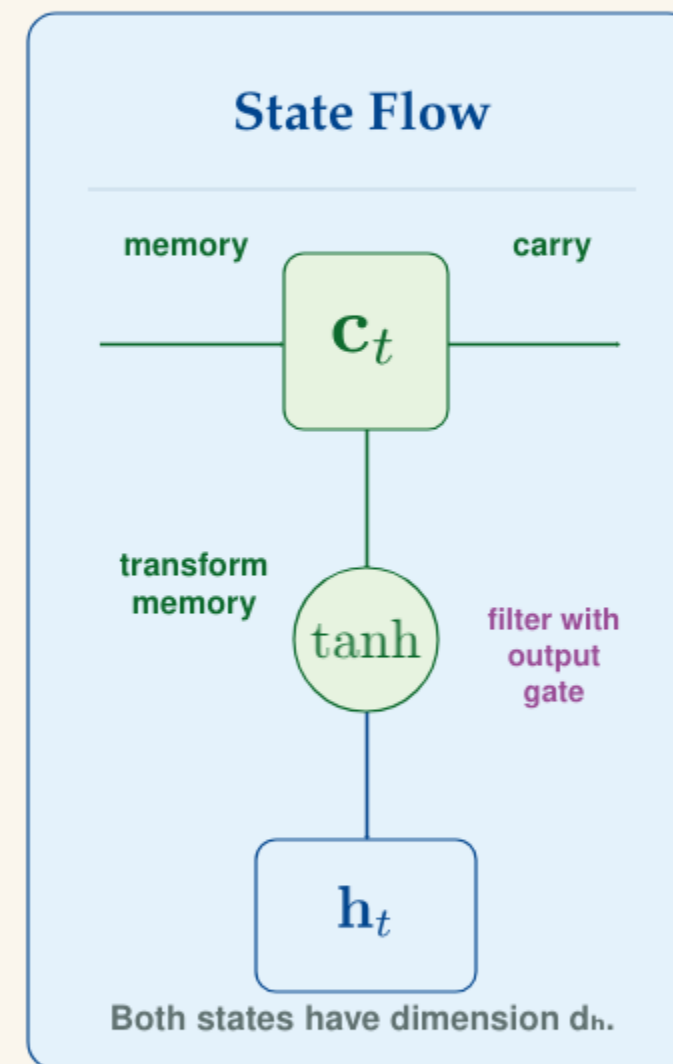
The reset gate acts inside the candidate path; the update gate acts at the final blend.

The Complete LSTM Cell

The gates and candidate jointly update the memory state and produce the visible hidden state.

ONE COMPLETE TIME STEP

Shared recurrent input	$\mathbf{q}_t = [\mathbf{h}_{t-1}; \mathbf{x}_t]$
Forget gate	$\Gamma_{f,t} = \sigma(W_f \mathbf{q}_t + \mathbf{b}_f)$
Update gate	$\Gamma_{u,t} = \sigma(W_u \mathbf{q}_t + \mathbf{b}_u)$
Candidate	$\tilde{\mathbf{c}}_t = \tanh(W_c \mathbf{q}_t + \mathbf{b}_c)$
Output gate	$\Gamma_{o,t} = \sigma(W_o \mathbf{q}_t + \mathbf{b}_o)$
Memory update	$\mathbf{c}_t = \Gamma_{f,t} \odot \mathbf{c}_{t-1} + \Gamma_{u,t} \odot \tilde{\mathbf{c}}_t$
Visible state	$\mathbf{h}_t = \Gamma_{o,t} \odot \tanh(\mathbf{c}_t)$

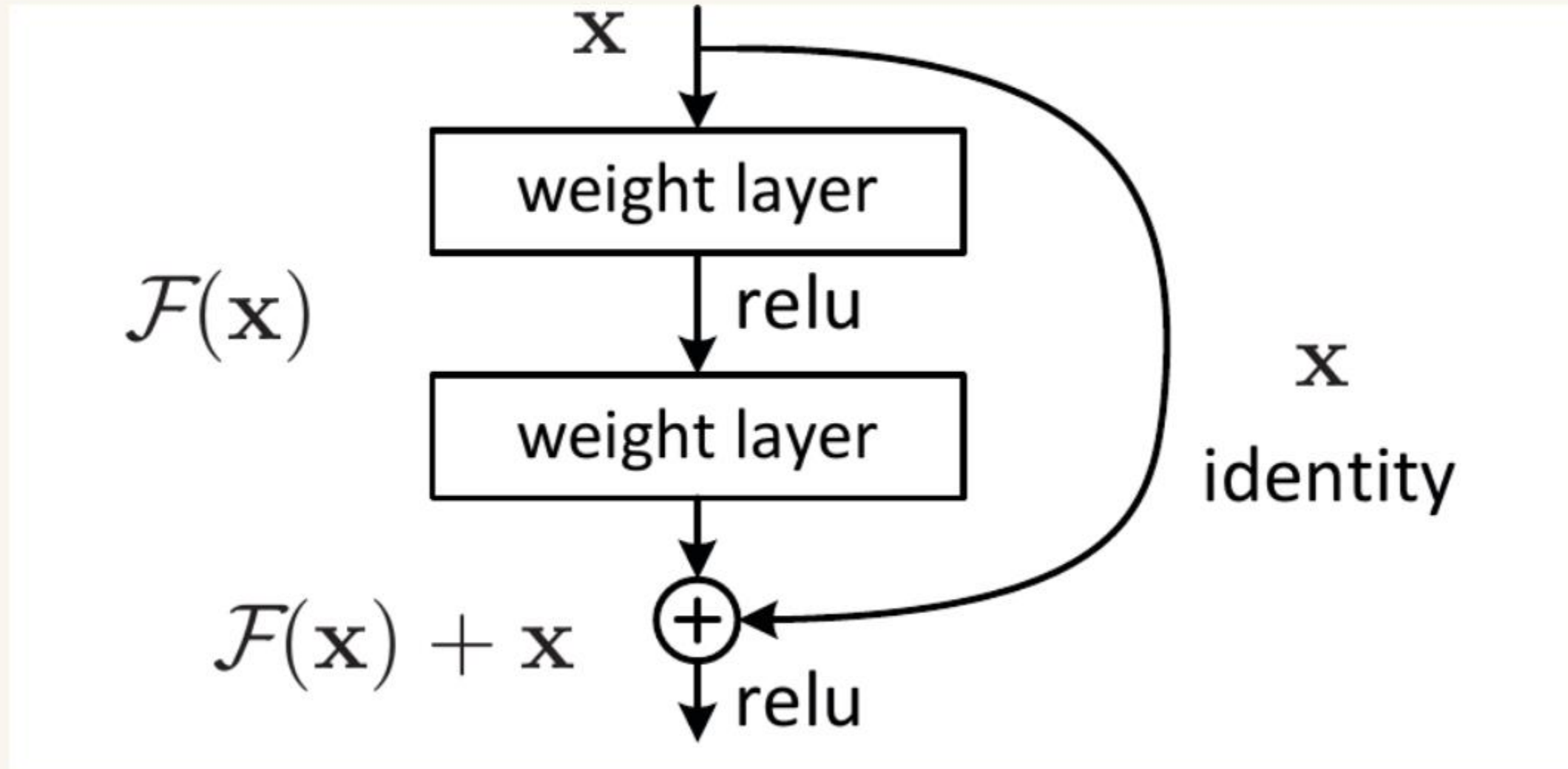


A scalar trace shows how the vector equations act on one memory coordinate.

Residual Learning: The Original Building Block

The shortcut carries the input directly to the addition node.

THE RESIDUAL BLOCK



Residual learning: a building block

Residual Connections and Gradient Flow

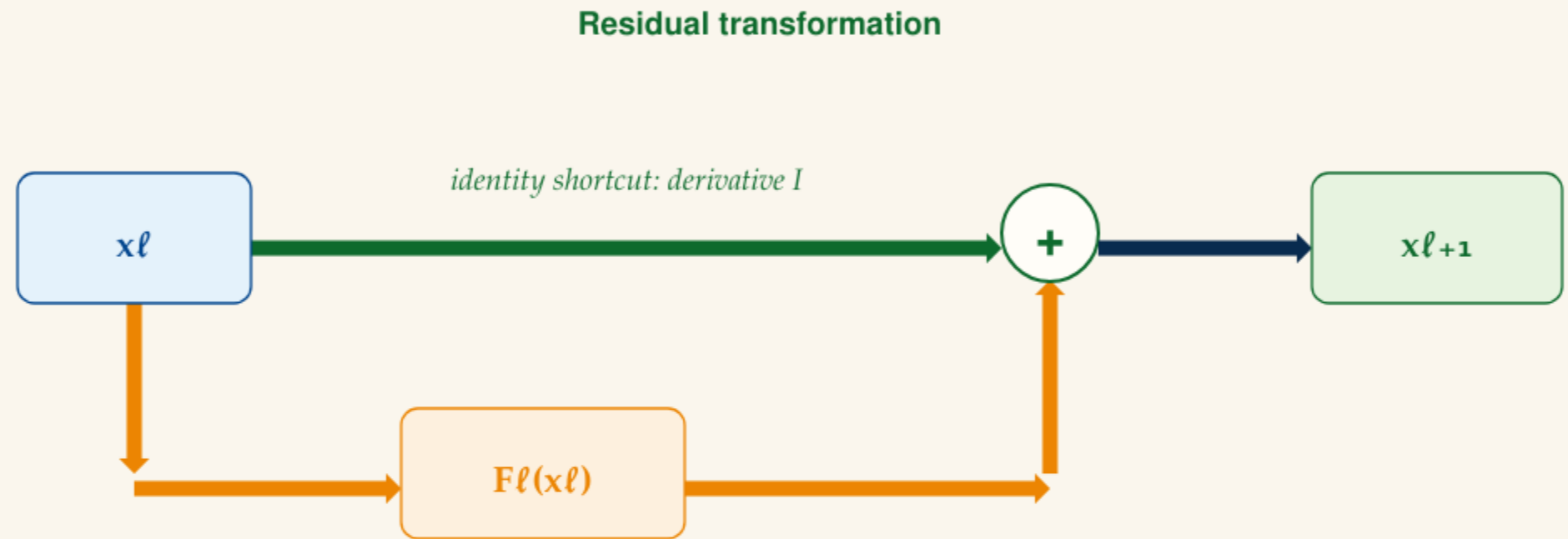
The additive shortcut contributes an identity term to the local derivative.

BACKPROPAGATION THROUGH ONE BLOCK

Plain transformation

$$\mathbf{x}_{\ell+1} = F_{\ell}(\mathbf{x}_{\ell})$$

$$\frac{\partial \mathbf{x}_{\ell+1}}{\partial \mathbf{x}_{\ell}} = J_{F_{\ell}}$$



$$\mathbf{x}_{\ell+1} = \mathbf{x}_{\ell} + F_{\ell}(\mathbf{x}_{\ell})$$

$$\frac{\partial \mathbf{x}_{\ell+1}}{\partial \mathbf{x}_{\ell}} = I + J_{F_{\ell}}$$

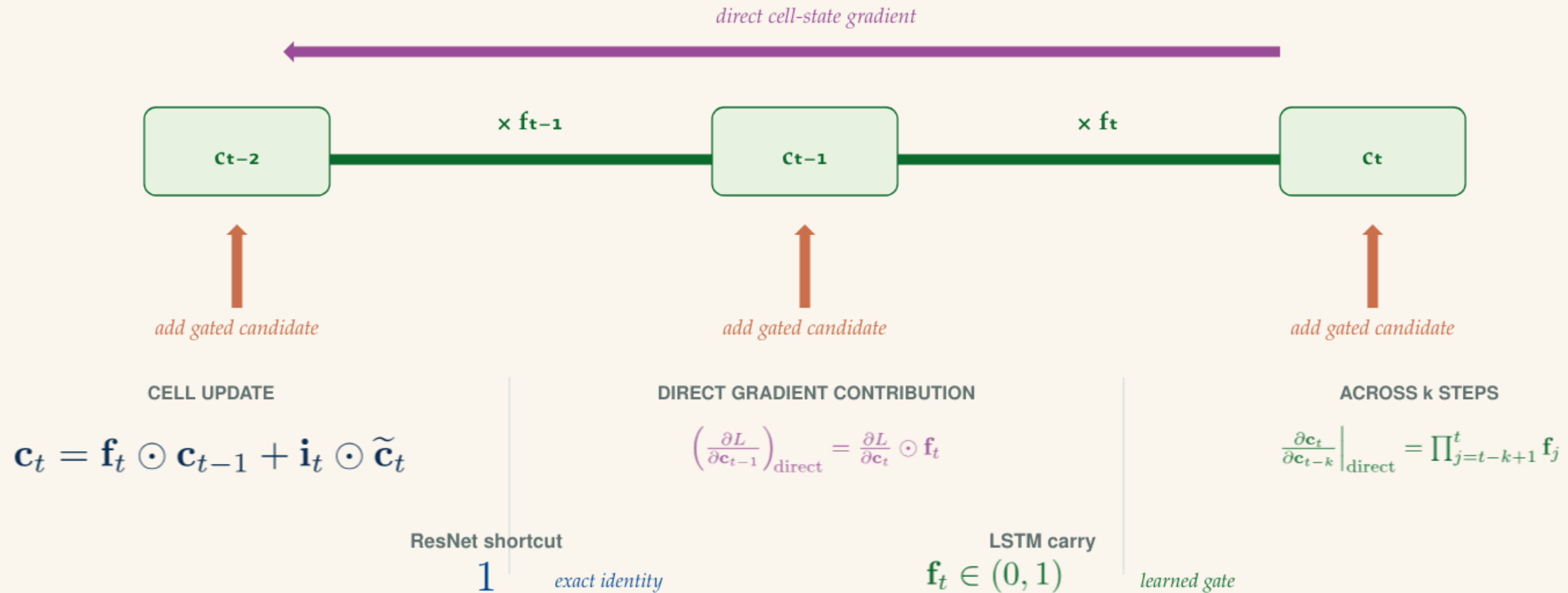
The identity term remains even when the learned branch has a small derivative.

Residual connections improve gradient flow. They do not guarantee perfect preservation.

The LSTM Cell State Is a Gated Residual Route Through Time

The additive cell update supplies a direct path, while the forget gate controls its strength.

THE ANALOGY, WITH THE IMPORTANT DIFFERENCE



When f_t is near 1, the direct gradient is preserved. Repeated values below 1 can still make it decay.

The Direct Gradient Path Through LSTM Memory

The forget gate controls how strongly each memory coordinate connects to its earlier value.

GRADIENT FLOW ALONG THE MEMORY-STATE PATH

Direct path across several time steps



One time step

$$\left. \frac{\partial c_t^{(j)}}{\partial c_{t-1}^{(j)}} \right|_{\text{direct}} = \Gamma_{f,t}^{(j)}$$

Across k steps

$$\left. \frac{\partial c_t^{(j)}}{\partial c_{t-k}^{(j)}} \right|_{\text{direct}} = \prod_{r=t-k+1}^t \Gamma_{f,r}^{(j)}$$

This isolates the direct additive route through memory.

The full derivative also includes indirect paths through the gates, candidate, and hidden state.

Retention Over 20 Steps

Constant forget-gate values

$$\Gamma_f^{(j)} = 0.99$$

$$0.99^{20} \approx 0.82$$

$$\Gamma_f^{(j)} = 0.50$$

$$0.50^{20} \approx 9.5 \times 10^{-7}$$

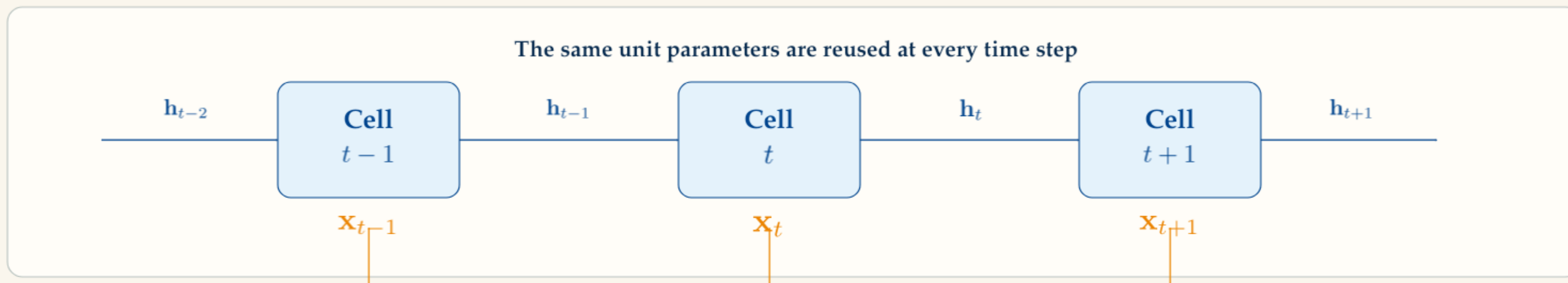
A gate near one can preserve a coordinate and its direct gradient for longer.

The complete LSTM cell can now be used as a recurrent unit inside several network architectures.

Recurrent Units and Recurrent Architectures

The recurrent update and the connections among time steps are separate design choices.

ONE RECURRENT UNIT REPEATED THROUGH TIME



THE RECURRENT UNIT CAN BE CHANGED

Vanilla RNN

$$\mathbf{h}_t = \tanh(W_x \mathbf{x}_t + W_h \mathbf{h}_{t-1} + \mathbf{b})$$

GRU

$$\mathbf{h}_t = \text{GRU}(\mathbf{x}_t, \mathbf{h}_{t-1})$$

LSTM

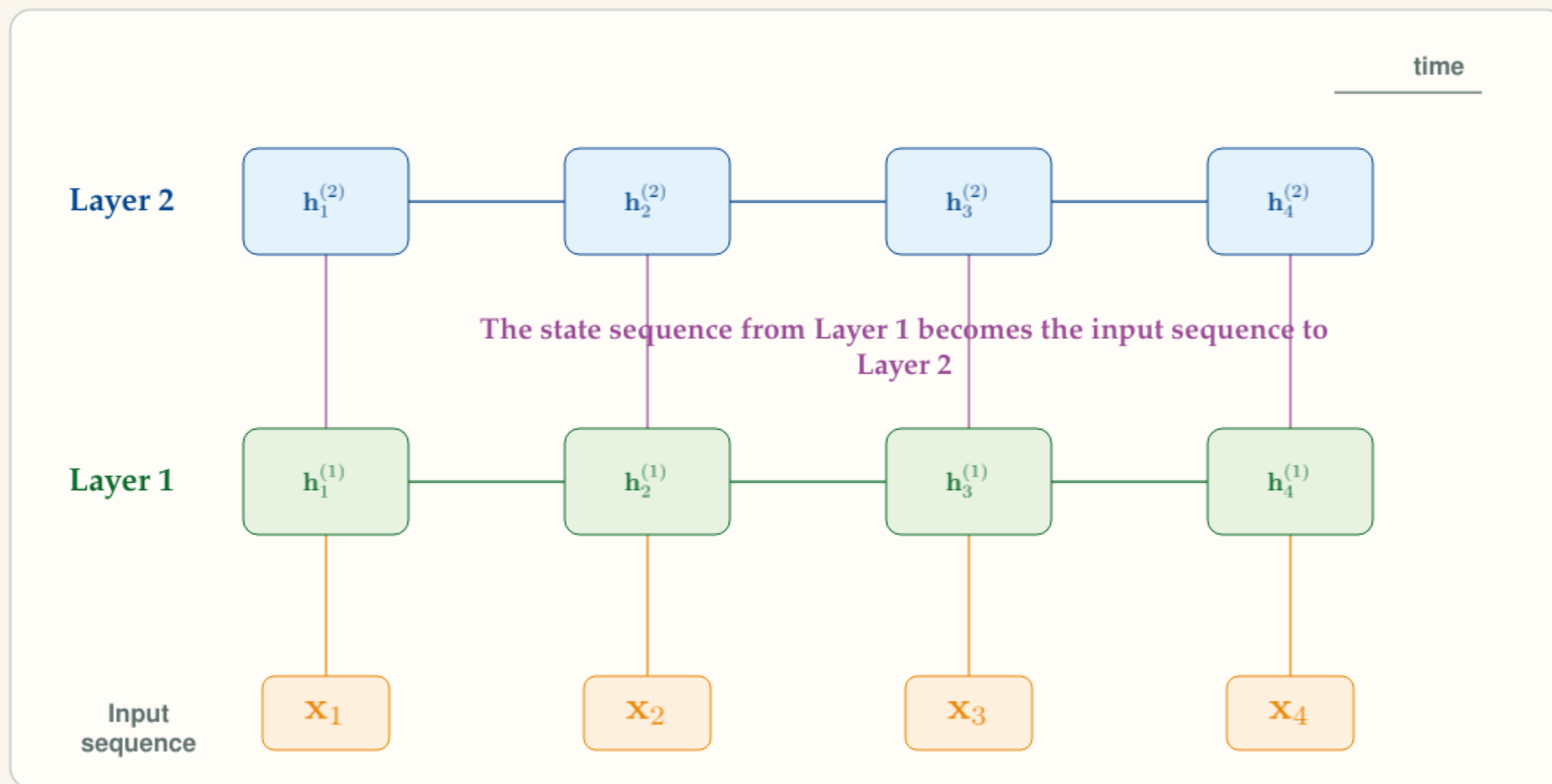
$$(\mathbf{h}_t, \mathbf{c}_t) = \text{LSTM}(\mathbf{x}_t, \mathbf{h}_{t-1}, \mathbf{c}_{t-1})$$

The first architectural extension adds recurrent depth.

Stacked Recurrent Networks

Each recurrent layer processes the sequence produced by the layer below.

RECURRENCE ACROSS TIME AND COMPUTATION THROUGH DEPTH



Layer Updates

$$h_t^{(1)} = \text{Cell}^{(1)}(x_t, h_{t-1}^{(1)})$$

$$h_t^{(2)} = \text{Cell}^{(2)}(h_t^{(1)}, h_{t-1}^{(2)})$$

Parameters are shared across time within each layer.

Different layers learn different parameter sets.

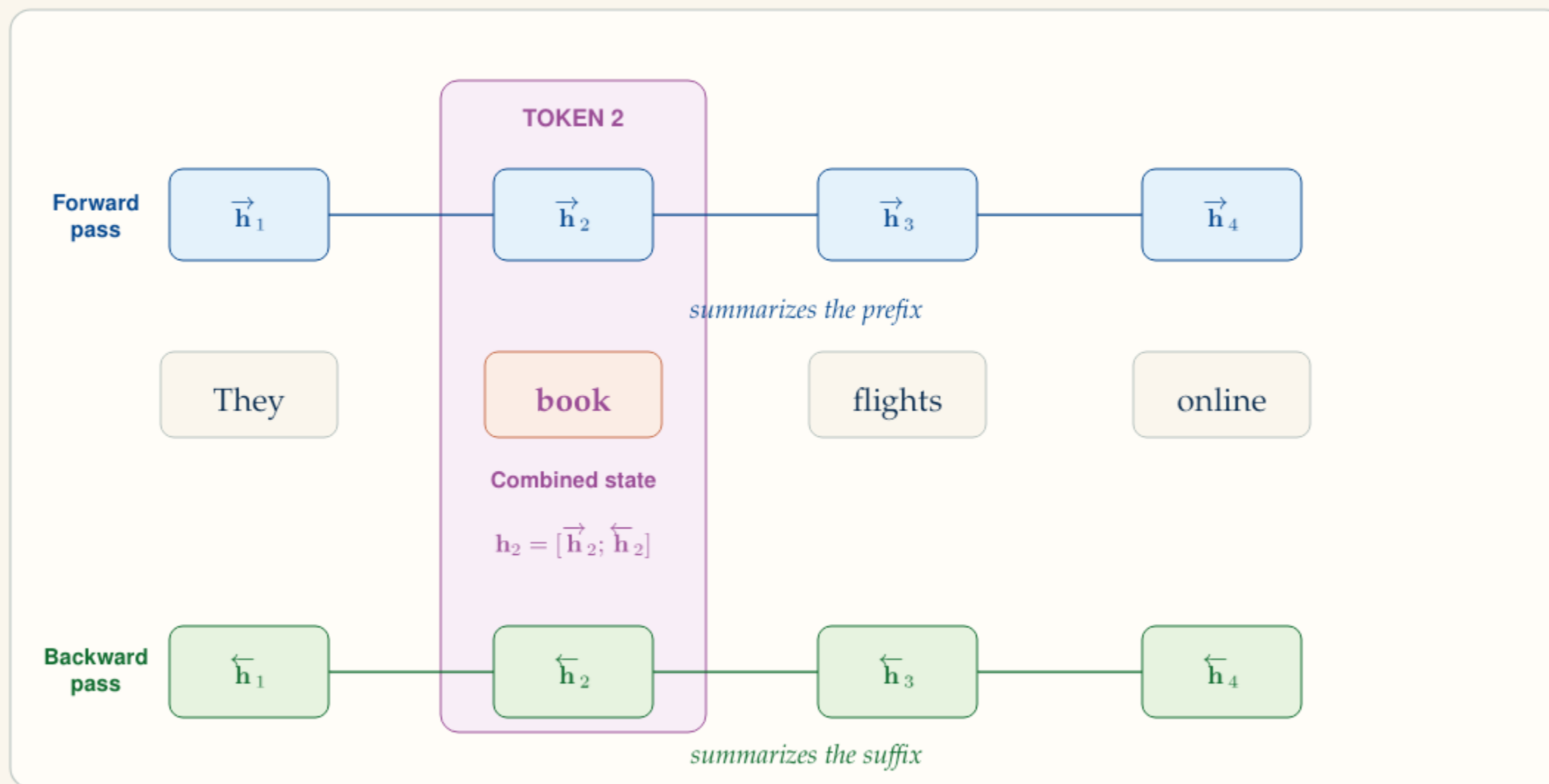
Cell = vanilla RNN, GRU, or LSTM

Stacking adds depth. Bidirectionality changes which positions provide context.

Bidirectional Recurrent Networks

A token receives one state from its left context and another from its right context.

CONTEXT FROM BOTH SIDES OF EACH TOKEN



State Equations

$$\vec{h}_t = \text{Cell}_f(\mathbf{x}_t, \vec{h}_{t-1})$$

$$\overleftarrow{h}_t = \text{Cell}_b(\mathbf{x}_t, \overleftarrow{h}_{t+1})$$

$$\mathbf{h}_t = [\vec{h}_t; \overleftarrow{h}_t] \in \mathbb{R}^{2d_h}$$

$$\hat{\mathbf{y}}_t = \text{softmax}(U\mathbf{h}_t + \mathbf{b})$$

book: VERB

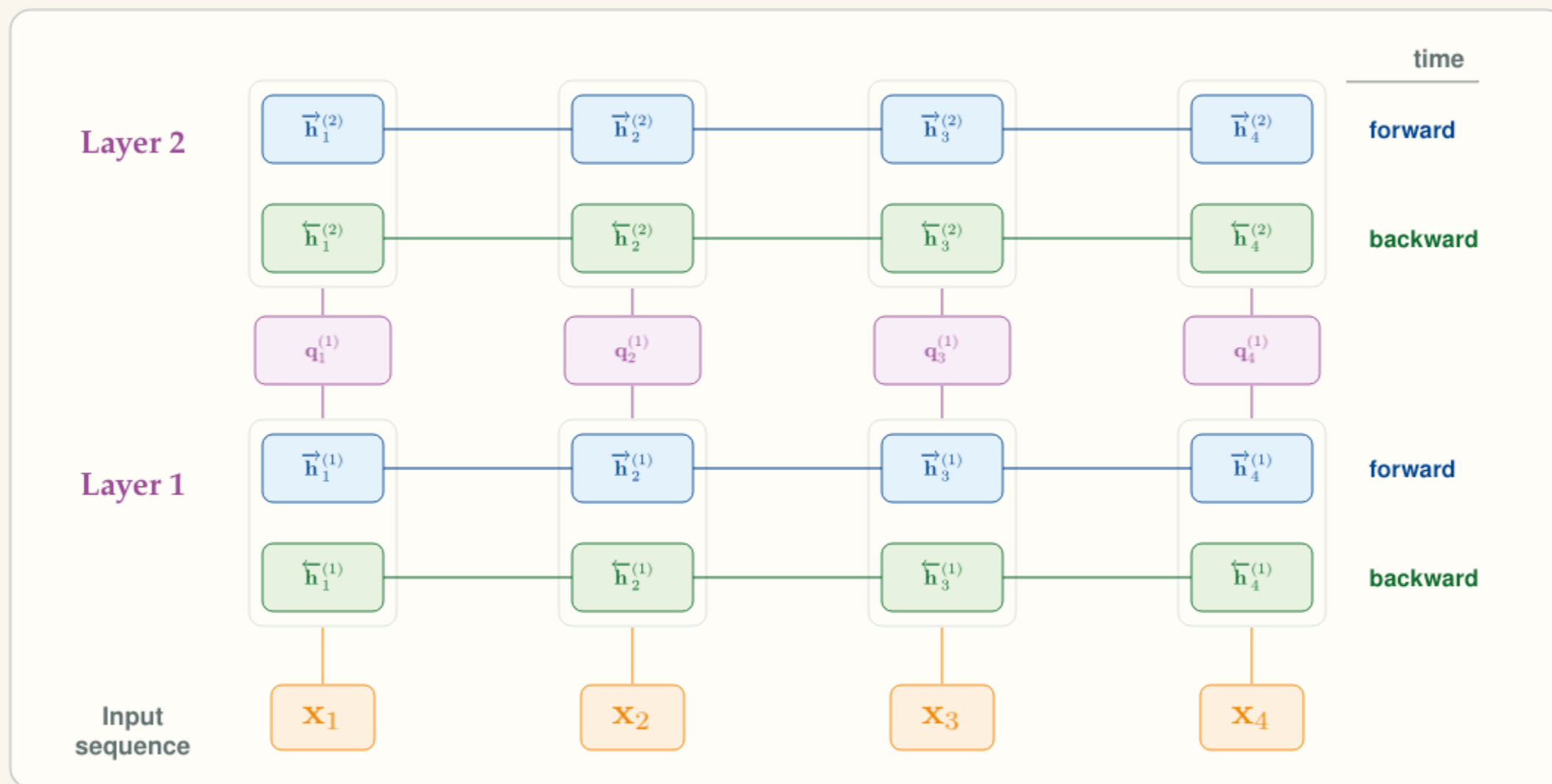
The backward pass requires the complete input sequence.

The combined state can drive a classifier at every token.

Stacked Bidirectional Recurrent Networks

Each upper recurrent layer processes the two-sided representation produced below.

TWO RECURRENT LAYERS, EACH RUNNING IN BOTH DIRECTIONS



Representation Passed Upward

$$q_t^{(1)} = [\vec{h}_t^{(1)}; \overleftarrow{h}_t^{(1)}]$$

This concatenated vector is the input to both directions in Layer 2.

$$\vec{h}_t^{(\ell)}, \overleftarrow{h}_t^{(\ell)} \in \mathbb{R}^d$$

$$q_t^{(\ell)} \in \mathbb{R}^{2d}$$

Upper-layer input matrices therefore accept a 2d-dimensional vector.

Cell = vanilla RNN, GRU, or LSTM

Each layer preserves recurrence in both directions and passes a 2d-dimensional representation upward.

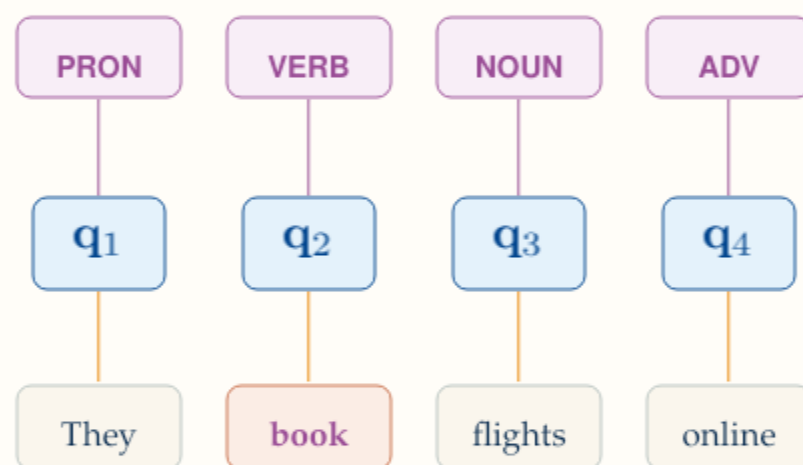
Recurrent Readouts for Different Prediction Tasks

The output layer and the available context depend on what the model predicts.

READOUTS ATTACHED TO RECURRENT STATES

Sequence labeling

One label for each input position



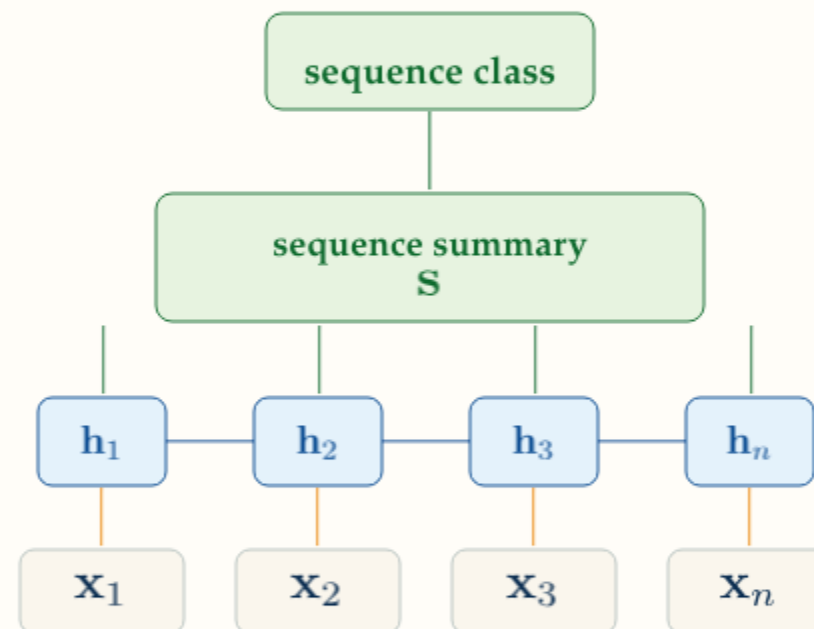
Bidirectional states may use the complete input

$$\hat{y}_t = \text{softmax}(W_{\text{tag}} \mathbf{q}_t + \mathbf{b})$$

PRON pronoun VERB verb
NOUN noun ADV adverb

Sequence classification

One prediction for the complete sequence

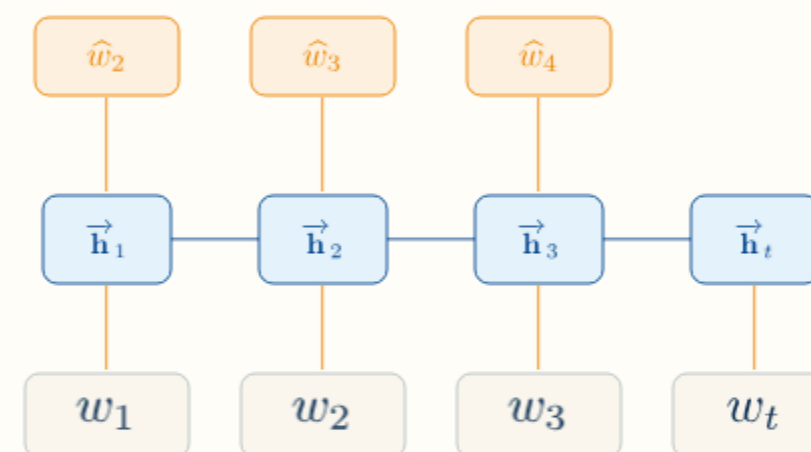


Final state or a pooled summary of the states

$$\hat{y} = \text{softmax}(W_{\text{cls}} \mathbf{s} + \mathbf{b})$$

Next-token prediction

Each state predicts the following token



Use only the forward state.
Future tokens must remain hidden.

$$\hat{y}_t = \text{softmax}(V \vec{h}_t + \mathbf{b})$$

Non-aligned output sequences require a separate decoder.

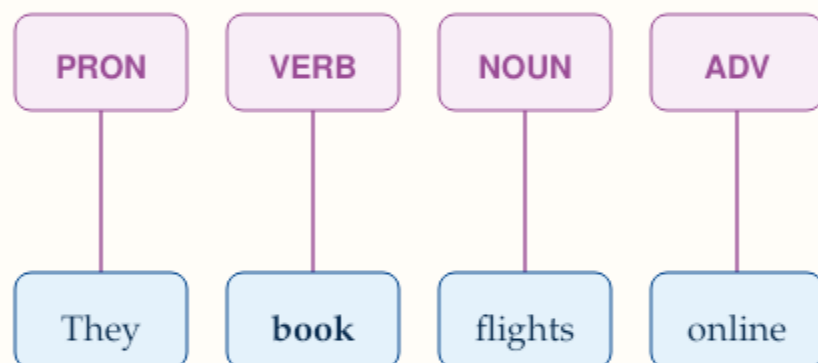
When Inputs and Outputs Do Not Align

Translation can change both sequence length and word order.

FROM ALIGNED READOUTS TO VARIABLE-LENGTH GENERATION

Aligned sequence labelling

Each input position has one output position.



The readout attaches to the recurrent state at the same position.

One shared input–output timeline

Translation

The target sequence has its own length and order.

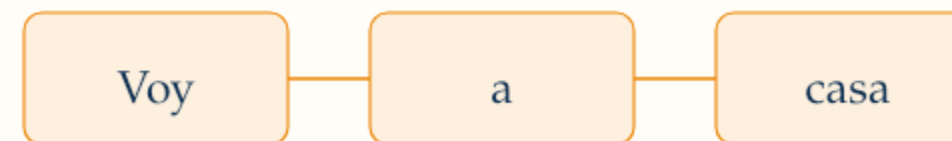
SOURCE $X_{1:n}$



$n \neq m$

Words may change position or combine across languages.

TARGET $Y_{1:m}$



The model must first represent the source, then generate the target on a separate recurrent timeline.

Google’s Neural Machine Translation System: Bridging the Gap between Human and Machine Translation

Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi
yonghui,schuster,zhifengc,qvl,mnorouzi@google.com

Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey,
Jeff Klingner, Apurva Shah, Melvin Johnson, Xiaobing Liu, Łukasz Kaiser,
Stephan Gouws, Yoshikiyo Kato, Taku Kudo, Hideto Kazawa, Keith Stevens,
George Kurian, Nishant Patil, Wei Wang, Cliff Young, Jason Smith, Jason Riesa,
Alex Rudnick, Oriol Vinyals, Greg Corrado, Macduff Hughes, Jeffrey Dean

Abstract

Neural Machine Translation (NMT) is an end-to-end learning approach for automated translation, with the potential to overcome many of the weaknesses of conventional phrase-based translation systems. Unfortunately, NMT systems are known to be computationally expensive both in training and in translation inference – sometimes prohibitively so in the case of very large data sets and large models. Several authors have also charged that NMT systems lack robustness, particularly when input sentences contain rare words. These issues have hindered NMT’s use in practical deployments and services, where both accuracy and speed are essential. In this work, we present GNMT, Google’s Neural Machine Translation system, which attempts to address many of these issues. Our model consists of a deep LSTM network with 8 encoder and 8 decoder layers using residual connections as well as attention connections from the decoder network to the encoder. To improve parallelism and therefore decrease training time, our attention mechanism connects the bottom layer of the decoder to the top layer of the encoder. To accelerate the final translation speed, we employ low-precision arithmetic during inference computations. To improve handling of rare words, we divide words into a limited set of common sub-word units (“wordpieces”) for both input and output. This method provides a good balance between the flexibility of “character”-delimited models and the efficiency of “word”-delimited models, naturally handles translation of rare words, and ultimately improves the overall accuracy of the system. Our beam search technique employs a length-normalization procedure and uses a coverage penalty, which encourages generation of an output sentence that is most likely to cover all the words in the source sentence. To directly optimize the translation BLEU scores, we consider refining the models by using reinforcement learning, but we found that the improvement in the BLEU scores did not reflect in the human evaluation. On the WMT’14 English-to-French and English-to-German benchmarks, GNMT achieves competitive results to state-of-the-art. Using a human side-by-side evaluation on a set of isolated simple sentences, it reduces translation errors by an average of 60% compared to Google’s phrase-based production system.

1 Introduction

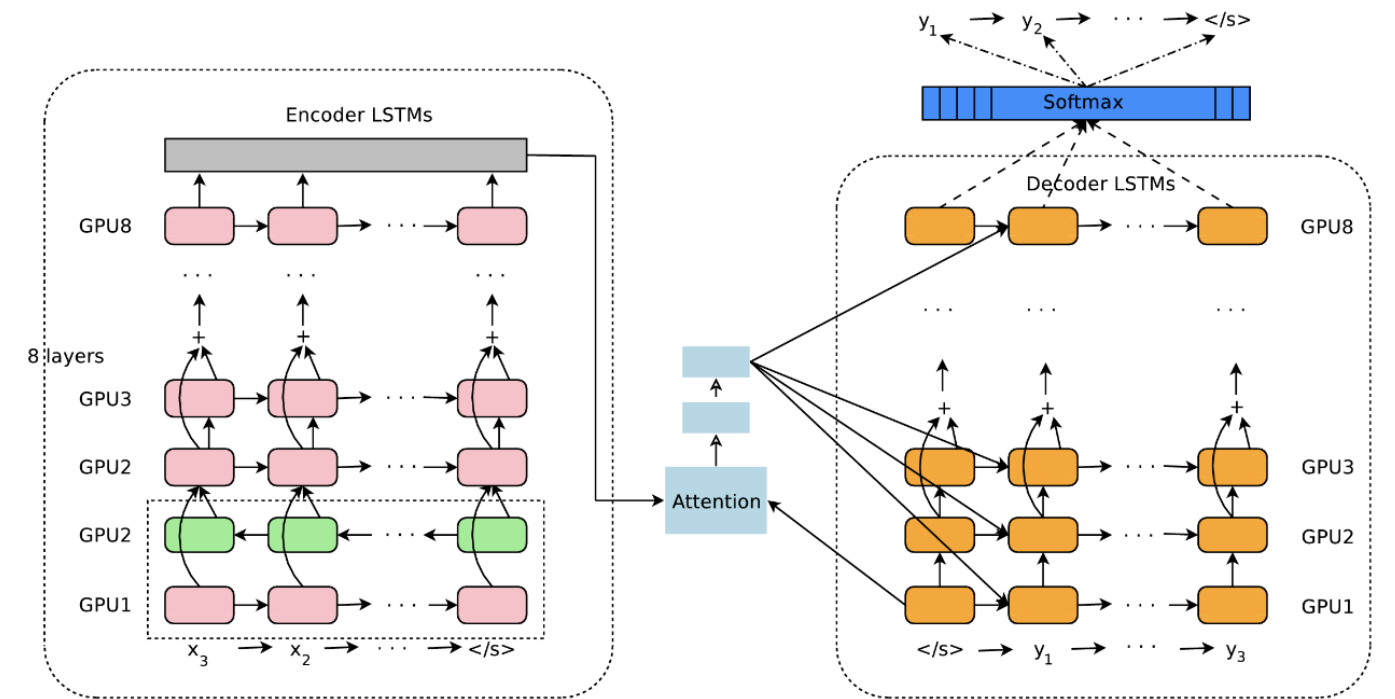


Figure 1: The model architecture of GNMT, Google’s Neural Machine Translation system. On the left is the encoder network, on the right is the decoder network, in the middle is the attention module. The bottom encoder layer is bi-directional: the pink nodes gather information from left to right while the green nodes gather information from right to left. The other layers of the encoder are uni-directional. Residual connections start from the layer third from the bottom in the encoder and decoder. The model is partitioned into multiple GPUs to speed up training. In our setup, we have 8 encoder LSTM layers (1 bi-directional layer and 7 uni-directional layers), and 8 decoder layers. With this setting, one model replica is partitioned 8-ways and is placed on 8 different GPUs typically belonging to one host machine. During training, the bottom bi-directional encoder layers compute in parallel first. Once both finish, the uni-directional encoder layers can start computing, each on a separate GPU. To retain as much parallelism as possible during running the decoder layers, we use the bottom decoder layer output only for obtaining recurrent attention context, which is sent directly to all the remaining decoder layers. The softmax layer is also partitioned and placed on multiple GPUs. Depending on the output vocabulary size we either have them run on the same GPUs as the encoder and decoder networks, or have them run on a separate set of dedicated GPUs.

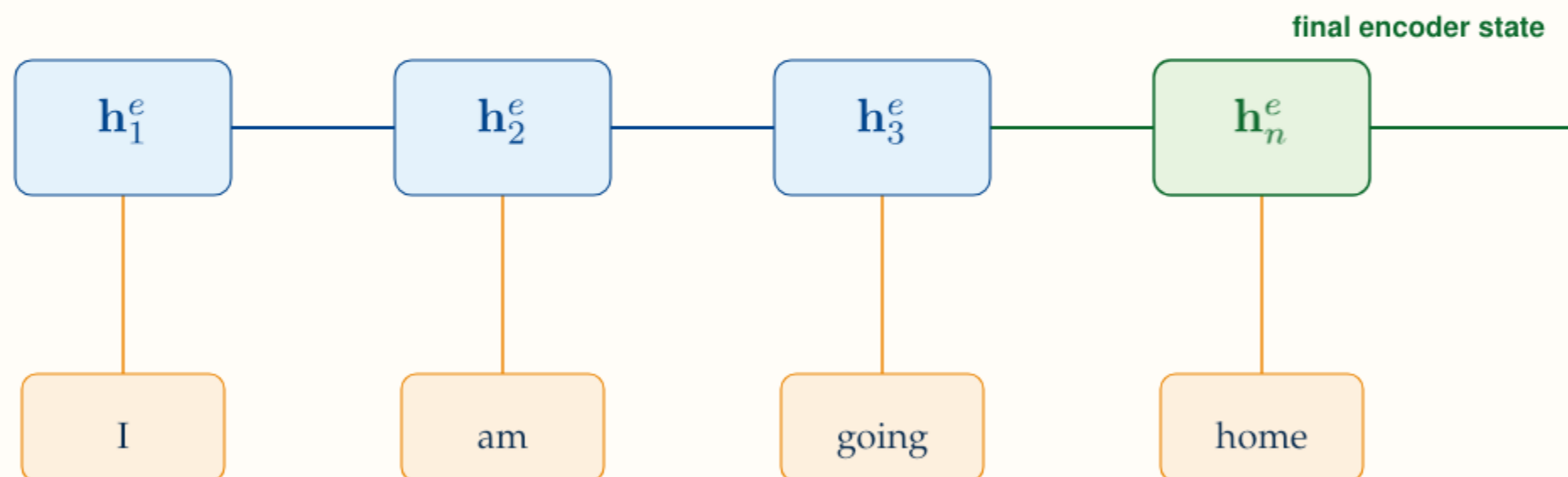
The Encoder Summarizes an Input Sequence

A recurrent encoder converts the source tokens into contextual hidden states.

ENCODING THE SOURCE SEQUENCE

The encoder creates one state at every source position

$$\mathbf{h}_i^e = f_{\text{enc}}(\mathbf{h}_{i-1}^e, E(x_i))$$



The final state carries the source history accumulated by the recurrent encoder.

Context Vector

Course notation

$$\mathbf{c}_{\text{ctx}} = \mathbf{h}_n^e$$

Jurafsky–Martin notation: c

A fixed-length summary
of the complete source
sequence

Encoder cell

Vanilla RNN, GRU or LSTM

A bidirectional encoder may combine two
terminal states.

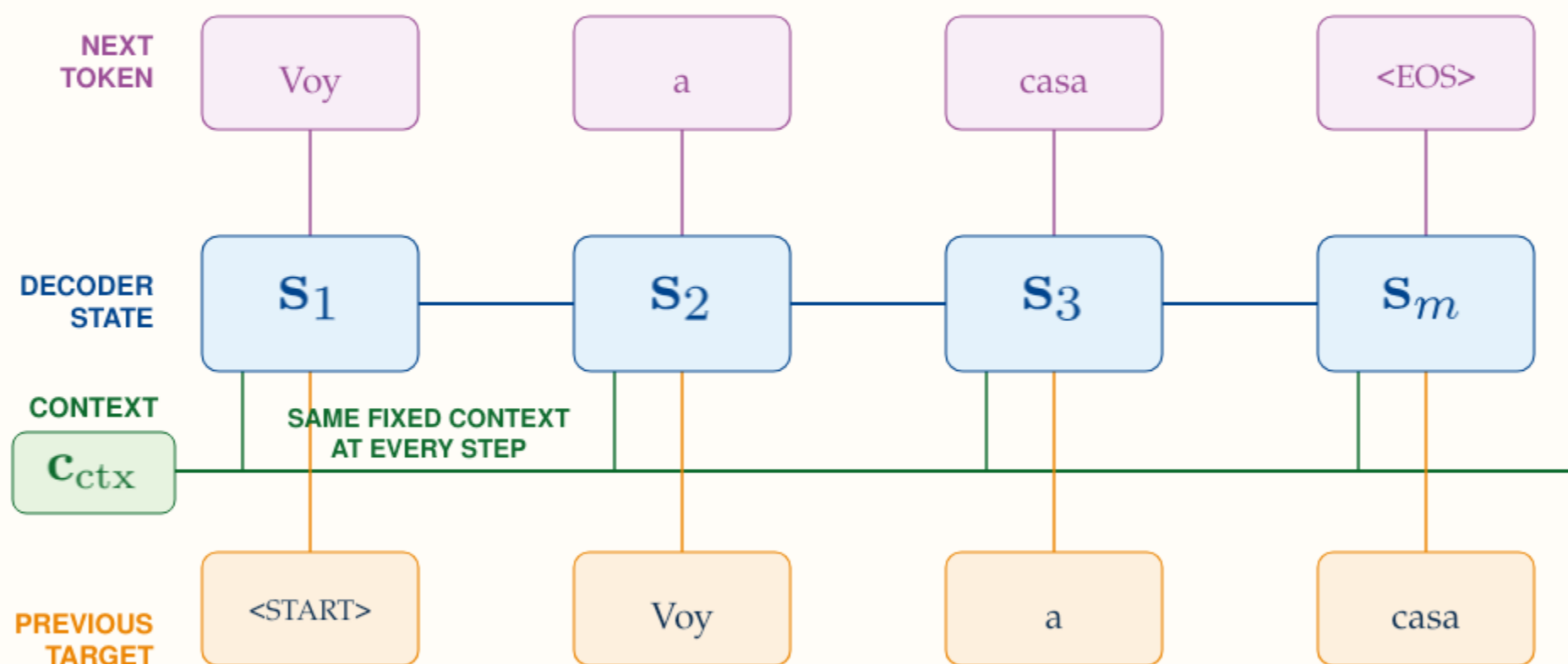
The context must now condition a second recurrent computation.

The Decoder Generates One Output at a Time

Each decoder state uses the previous target token and the encoder context.

AUTOREGRESSIVE TARGET GENERATION

The target sequence has its own recurrent timeline



The output at one step becomes the previous-token input at the next step.

One Decoder Step

State update

$$s_t = g_{dec}(s_{t-1}, E(y_{t-1}), c_{ctx})$$

Vocabulary distribution

$$p(y_t | y_{<t}, \mathbf{x}) = \text{softmax}(V s_t + \mathbf{b})$$

<START> begins generation.
<EOS> ends the target sequence.

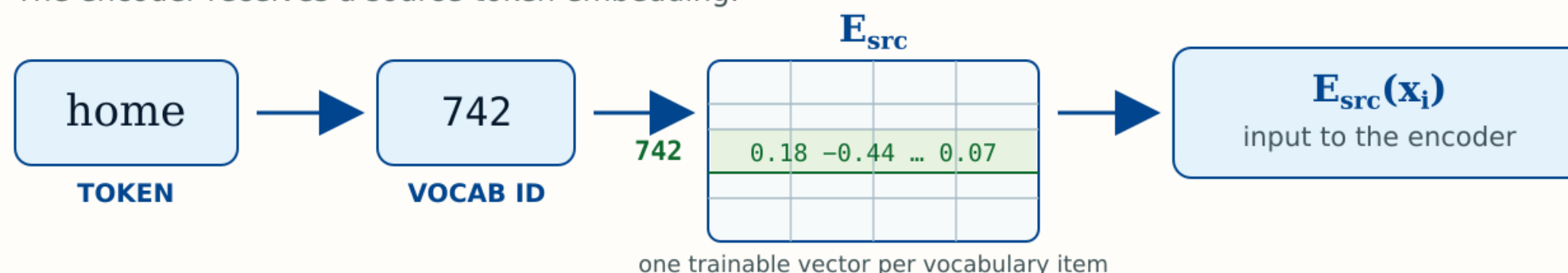
Where Does $E(\cdot)$ Come From?

A token ID selects one row of a learned embedding matrix.

FROM DISCRETE TOKENS TO CONTINUOUS VECTORS

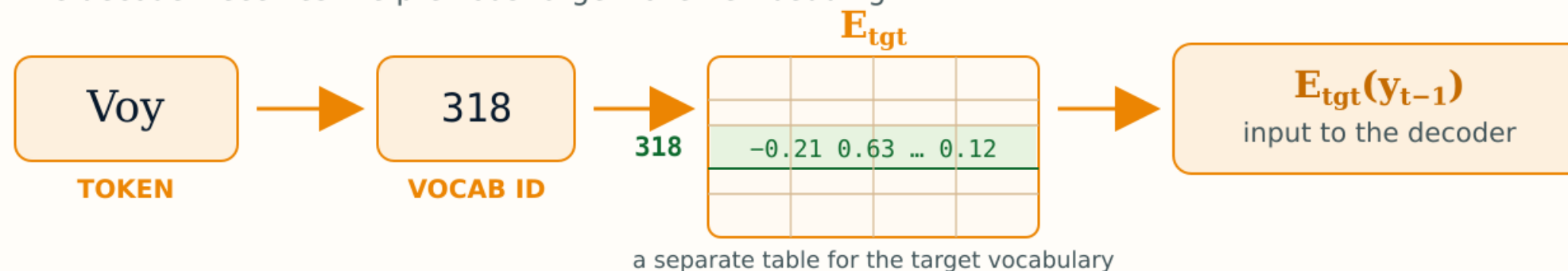
SOURCE INPUT

The encoder receives a source-token embedding.



PREVIOUS TARGET

The decoder receives the previous target-token embedding.



How are the rows learned?

INITIAL VALUES

Usually random

Optionally pretrained

Word2Vec or GloVe

DURING SEQUENCE-MODEL TRAINING

Translation loss

backpropagates into
both embedding tables.

The tables remain trainable.

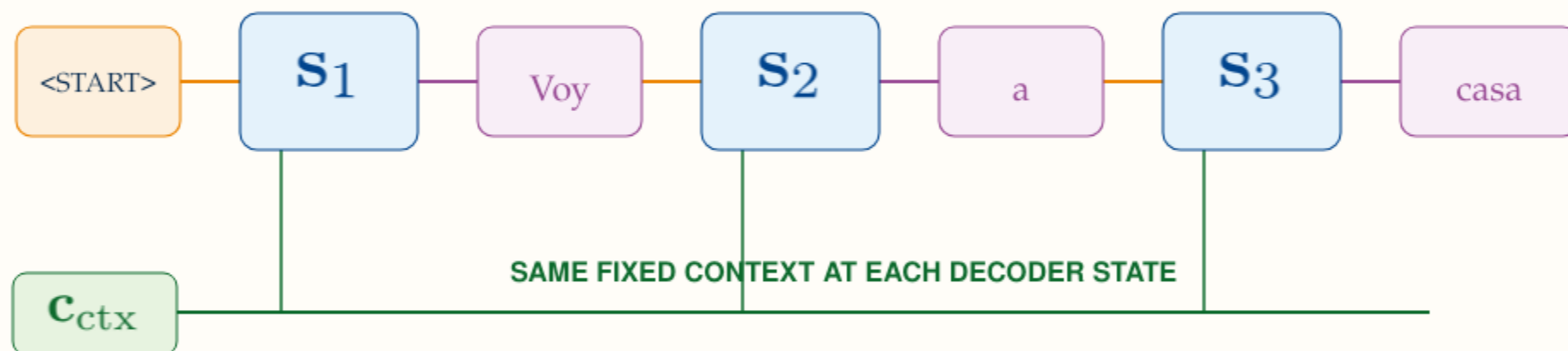
$E(\cdot)$ is usually a trainable lookup table, learned jointly with the sequence model.

Autoregressive Inference Uses Its Own Predictions

After <START>, each selected token becomes input to the following decoder step.

GENERATION WITHOUT A GOLD TARGET

Each selected token becomes part of the next decoder input



Only the model's selected tokens are available after <START>.
An early mistake changes the later decoder states.

One Inference Step

State update

$$\mathbf{s}_t = g_{\text{dec}}(\mathbf{s}_{t-1}, E(\hat{y}_{t-1}), \mathbf{c}_{\text{ctx}})$$

Vocabulary distribution

$$\mathbf{p}_t = \text{softmax}(V\mathbf{s}_t + \mathbf{b})$$

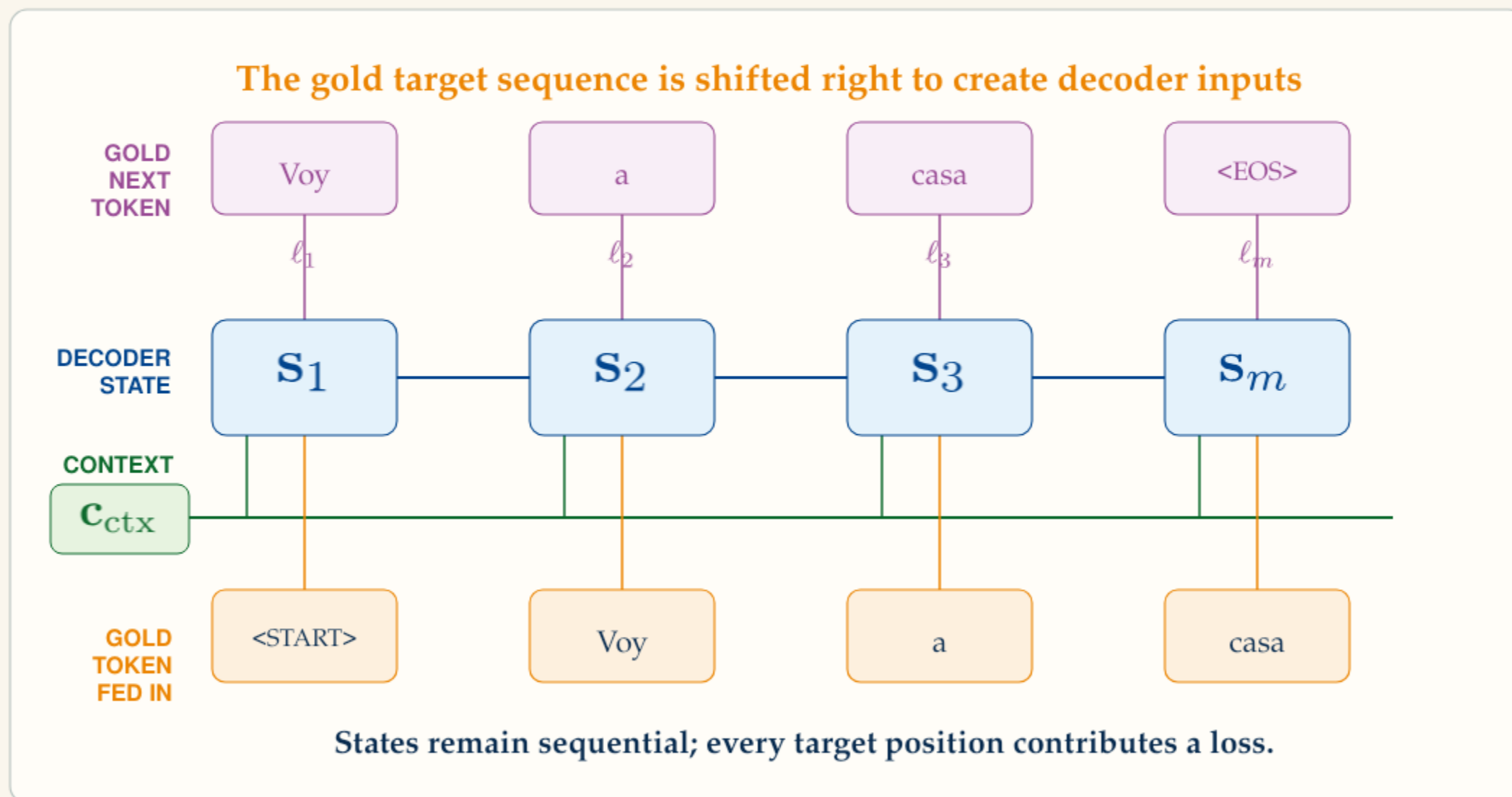
Select one token from the distribution.
<EOS> terminates generation.

The decoder can use only the source information contained in the fixed context.

Teacher Forcing During Training

The observed previous target token enters the decoder at every training step.

TRAINING THE RECURRENT DECODER



One Training Step

Teacher-forced state

$$s_t = g_{\text{dec}}(s_{t-1}, E(y_{t-1}^*), c_{\text{ctx}})$$

Token loss

$$\ell_t = -\log p(y_t^* | y_{<t}^*, \mathbf{x})$$

The decoder receives the correct previous token, even when its own prediction was wrong.

At inference time, the observed target sequence is unavailable.